

PRO-CUA: Process-Reward Optimization for Computer Use Agents

Anonymous ACL submission

Abstract

Computer use agents (CUAs) have shown strong potential for automating complex digital workflows, yet their training remains constrained by costly live environment interaction and limited high-quality supervision. Existing filtered behavior cloning pipelines suffer from imitation bottlenecks, including distribution shift from the expert demonstration and the absence of negative learning signals. Meanwhile, standard trajectory-level reinforcement learning struggles with sparse rewards, ambiguous credit assignment, and high infrastructure costs for long-horizon GUI interaction. In this work, we propose PRO-CUA, a process-reward optimization framework for training CUAs with iterative step-level reinforcement learning. PRO-CUA decouples on-policy environment interaction from policy optimization: the current policy collects states through live rollouts, generates diverse candidate actions for each state, receives step-level feedback from a process reward model (PRM), and is optimized with group-relative advantages. This design enables dense and flexible credit assignment without relying on golden answers or offline expert trajectories, while reducing distribution shift by training on the agent’s own execution states. Experiments on live web benchmarks demonstrate the effectiveness of PRO-CUA and the reliability of PRM-guided step-level training.

1 Introduction

Driven by rapid breakthroughs in multimodal reasoning, autonomous agents are evolving into economically valuable digital coworkers. Computer use agents (CUAs) (OpenAI, 2025; Anthropic, 2024; Agashe et al., 2025; Qin et al., 2025; Wang et al., 2025a,c) have proven highly capable of seamlessly automating complex, open-ended workflows. By perceiving visual interfaces and executing sequential plans, these agents natively operate across diverse digital ecosystems, including web

browsers (Deng et al., 2023; He et al., 2024a; Xue et al., 2025) and desktop environments (Xie et al., 2024; Bonatti et al., 2025; Wu et al., 2025). Despite their immense commercial value and recent capability leaps, creating reliably generalized CUAs remains fundamentally bottlenecked by how they are trained. Specifically, researchers are constrained by two interlocking challenges: the prohibitive latency and computational cost of interacting with live GUI environments, and an acute scarcity of high-quality training data.

The most intuitive and prevailing approach to training CUAs is filtered behavior cloning (FBC) from expert demonstrations (Bai et al., 2024; Xu et al., 2025; He et al., 2024b; Shen et al., 2025; He et al., 2026). However, FBC inherently suffers from imitation bottlenecks: it over-penalizes reasoning diversity, disproportionately overfits to easy tasks, and lacks negative learning signals for agents to learn from mistakes. To overcome these limitations, reinforcement learning (RL) offers a principled alternative. Yet, applying standard, *trajectory-level* RL to CUAs introduces optimization and infrastructural difficulties. In long-horizon computer use tasks, receiving a single sparse reward at task completion makes step-wise credit assignment highly ambiguous, so the agent cannot deduce which specific action among dozens caused the ultimate failure. Furthermore, standard synchronous RL frameworks such as verl (Sheng et al., 2025) are computationally ill-equipped for multi-turn agent workflows. The latency of live GUI execution combined with the compounding memory costs of token-heavy image contexts renders trajectory-level optimization especially challenging.

To bypass the sparse reward problem of trajectory-level learning, recent efforts have shifted toward *step-level* RL paradigms (Luo et al., 2025; Yang et al., 2026). While step-wise optimization makes credit assignment tractable, existing methods remain constrained by their reward design and

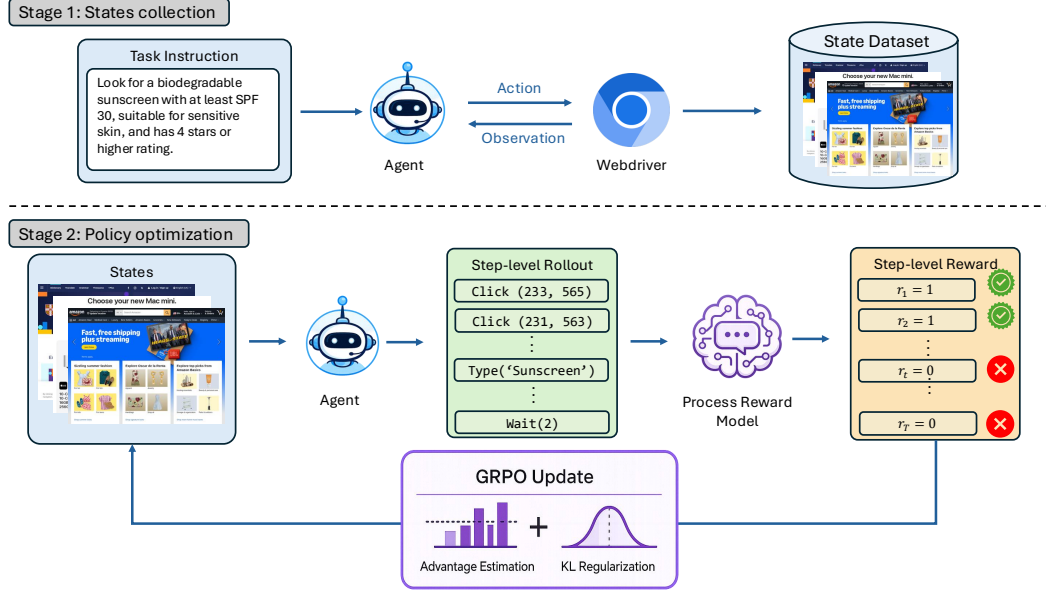


Figure 1: **Overview of the PRO-CUA pipeline.** PRO-CUA alternates between two stages across multiple training iterations. In **Stage 1**, the current policy interacts with the live environment to collect on-policy states. In **Stage 2**, policy optimization is performed without further environment interaction through three steps: **i) Step-level generation:** The agent samples multiple candidate actions for each collected state; **ii) PRM grading:** A process reward model assigns binary step-level rewards; and **iii) GRPO update:** The policy is optimized using group-relative advantages. The updated policy is then used for the next round of on-policy state collection.

data collection mechanisms. They predominantly rely on *rule-based rewards* that require exact-match accuracy against a *golden answer*, a strict requirement that drastically limits the scalability of viable training data. Moreover, these pipelines are primarily *off-policy*, optimizing over states collected by a stronger teacher model rather than the target policy itself. Because an agent’s actions sequentially alter future GUI observations, this off-policy collection introduces compounding distribution shift (Ross et al., 2011). The offline training data can diverge from the actual, suboptimal states the agent will encounter, resulting in brittle performance and an inability to recover from mistakes.

To tackle the dual challenges of reward scarcity and off-policy distribution shift, we propose an on-policy self-evolvment framework termed PRO-CUA (Process-Reward Optimization for Computer Use Agents). As shown in Figure 1, PRO-CUA alternates between two stages across multiple training iterations. In Stage 1, the current policy interacts with the live environment at an elevated sampling temperature to collect on-policy states. In Stage 2, policy optimization is performed without further environment interaction: for each collected state, the agent samples G diverse thought-action pairs, a Process Reward Model (PRM) assigns binary step-level rewards, and the policy is

updated with GRPO. This design decouples live environment interaction from policy optimization, allowing each stage to run with infrastructure tailored to its own computational profile while training the agent on its own execution states.

Empirically, we verify the effectiveness of the PRO-CUA pipeline on live web benchmarks, including WebVoyager (He et al., 2024a), Mind2Web-Live (Pan et al., 2024) and Online Mind2Web (Xue et al., 2025). We further demonstrate the reliability of our PRM grading pipeline and the effective data utilization of our training approach.

Overall, our PRO-CUA framework provides a scalable pipeline for training computer use agents. In summary, our main contributions are: **i) Tailored CUA infrastructure design:** We decouple environment interaction and model training, avoiding the system challenges to simultaneously perform agent rollout, environment interaction and policy optimization. **ii) On-policy state collection:** We eliminate the reliance on offline expert demonstrations, and enable the agent to learn from its own execution distribution. **iii) Dense and flexible credit assignment via PRMs:** We transition from sparse trajectory-level rewards to PRM-graded step-level GRPO. This provides fine-grained supervision without the requirement of collecting golden answers from expert demonstration.

2 Preliminaries

Computer use agents (CUAs) CUAs take sequential steps to interact with a graphical user interface (GUI) to complete a task defined in the task instruction. Following the ReAct (Yao et al., 2023) paradigm, agents typically generate interleaved thoughts and actions, explicitly externalizing their reasoning process to improve task execution. At any given step n , the agent receives a state context x_n , which comprises the instruction $\mathcal{I}$, the historical sequence of past thoughts and actions $\{(t_i, a_i)\}_{i=1}^{n-1}$, and a truncated window of the w most recent visual observations (screenshots) $\{o_{n-j}\}_{j=0}^{w-1}$. Truncation is applied exclusively to the visual inputs to manage the prohibitive token costs associated with multimodal contexts. In this work, we set $w = 1$, so the agent observes only the most recent screenshot, which is memory-efficient and has been shown to be sufficient for GUI perception (Qin et al., 2025). The agent’s objective is to learn a policy $\pi_\theta(t_n, a_n \mid x_n)$ that iteratively selects actions leading to successful task completion.

Filtered Behavior Cloning (FBC) Currently, the most prevalent training paradigm for CUAs is FBC, which is conceptually equivalent to Rejection Sampling Fine-Tuning (RFT). In this approach, a dataset of candidate trajectories is filtered to retain only those that result in a successful final outcome, forming a curated dataset $\mathcal{D}_{\text{succ}}$. The policy is then optimized via standard supervised fine-tuning (SFT) to maximize the log-likelihood of the expert thoughts and actions:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{\tau \sim \mathcal{D}_{\text{succ}}} \left[\sum_{n=1}^{|\tau|} \log \pi_\theta(t_n, a_n \mid x_n) \right].$$

Reinforcement Learning (RL) While FBC optimizes the likelihood of both the thought tokens and action tokens, RL only requires a reward signal for the generated action. This distinction is important for CUAs: intermediate thoughts can be long and diverse, whereas task progress is ultimately determined by the executed action. RL therefore avoids forcing the policy to reproduce the reference reasoning trace, and instead reinforces any generation that leads to a rewarded action.

In single-turn domains like mathematical reasoning, GRPO relies only on a sparse outcome reward (e.g., verifying the answer). However, this trajectory-level approach is often suboptimal for long-horizon computer use

tasks due to credit assignment challenges. To address this issue, recent works deploy *step-level* RL. Similar to SFT, given step context $x = \{\mathcal{I}, \{t_{n-i}, a_{n-i}\}_{i=1}^{n-1}, \{o_{n-i}\}_{i=1}^w\}$, GRPO samples a group of G candidate thought-action pairs $\{t_k, a_k\}_{k=1}^G \sim \pi_\theta(\cdot \mid x)$ and compute the reward $r_k = \mathcal{R}(x, a_k)$, where $\mathcal{R}$ is predominantly a rule-based reward based on a comparison with *the golden answer* obtained through expert demonstration, which measures accuracy of the action based on the action type, the input text and the coordinates the agent interacts with (detailed formulation in Appendix C). The overall objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_{\substack{x \sim D \\ \{a_k\}_{k=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)}} \left[\frac{1}{G} \sum_{k=1}^G \min \left(\rho_k \hat{A}_k, \text{clip}(\rho_k, 1 - \epsilon, 1 + \epsilon) \hat{A}_k \right) - \beta \cdot \text{KL}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)) \right] \quad (1)$$

where $\rho_k = \frac{\pi_\theta(a_k \mid x)}{\pi_{\theta_{\text{old}}}(a_k \mid x)}$ represents the importance sampling ratio, $\hat{A}_k$ is the relative advantage computed within the sampled group, and β controls the KL divergence penalty against the reference model π_{ref} to prevent policy collapse.

3 PRO-CUA

3.1 On-policy State Collection

Prior works primarily rely on distillation from strong teacher models or human demonstrations (He et al., 2026; Yang et al., 2026; Luo et al., 2025). However, these expert trajectories suffer from severe distribution shift: they fail to represent the sub-optimal states a developing agent actually encounters. Because an agent’s primitive actions alter subsequent observations, it inevitably drifts into catastrophic or out-of-distribution states (e.g., stuck websites) that are entirely absent from expert data.

While online RL offers a principled solution by allowing the agent to explore via its own policy, it introduces severe infrastructural bottlenecks in the CUA setting. Synchronizing high-throughput LLM inference (e.g., vLLM (Kwon et al., 2023)), high-latency web browser interactions, and dedicated training frameworks (e.g., verl (Sheng et al., 2025)) into a single loop results in prohibitive latency, I/O overhead, and hardware idling.

To overcome these bottlenecks while preserving the necessity of on-policy exploration, we propose a decoupled state collection paradigm. As illustrated in Stage 1 of Figure 1, we disentangle the slow environment interactions from the compute-

heavy policy optimization loop. We deploy the current agent policy to interact with live environments at an elevated sampling temperature, encouraging the discovery of diverse paths. During the agent rollout, we continuously harvest these exploratory trajectories, logging the task instructions, visual observations, and action histories into a state dataset $\mathcal{D}_{\text{state}}$. Formally, let $\mathcal{T}$ represent a set of collected trajectories (which may include both successful and failed rollouts). A single trajectory $\tau \in \mathcal{T}$ of length $|\tau|$ is composed of a sequence of step-level interactions. At each step $n \in \{1, \dots, |\tau|\}$, the environment and the agent’s history dictate a state context $x_n^{(\tau)}$, formally defined as the tuple:

$$\mathcal{D}_{\text{state}} = \bigcup_{\tau \in \mathcal{T}} \bigcup_{n=1}^{|\tau|} \{x_n^{(\tau)}\},$$

where $x_n^{(\tau)} = (\mathcal{I}^{(\tau)}, \{(t_i, a_i)\}_{i=1}^{n-1}, \{o_{n-j}\}_{j=0}^{w-1})$. The dataset will be further employed in the next stage to optimize the policy.

This decoupled mechanism solves two critical problems. First, it allows for highly parallelized, scalable data generation without bottlenecking the training hardware. Second, by populating the dataset with the agent’s own exploratory rollouts rather than expert demonstrations, the policy is forced to confront its own mistakes, natively generating the failure states required to learn error-recovery during the subsequent optimization stage.

3.2 Step-level Rollout

Given the on-policy state dataset collected in the previous stage, PRO-CUA proceeds to the step-level rollout process. In each training iteration, the agent samples states from $\mathcal{D}_{\text{state}}$ and, for each state, generates G candidate thought-action pairs. These candidates are newly sampled from the current policy and are not the actions originally executed during state collection. Thus, the collected trajectories serve only to provide the state distribution encountered by the agent, rather than reference demonstrations for imitation.

Crucially, the candidate actions are *not executed in the live environment*. This avoids repeatedly invoking costly browser interactions inside the optimization loop, but also means that the resulting state transitions are not directly observed. We therefore defer evaluation to the PRM, as described in the next section.

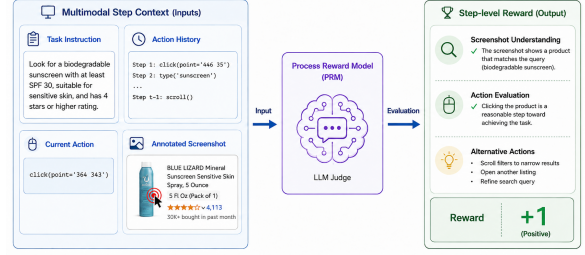


Figure 2: **Process Reward Model (PRM) grading pipeline.** The PRM receives a multimodal step context comprising the task instruction, the agent’s action history, the proposed current action, and an annotated screenshot. For readability, the figure shows a zoomed-in crop, while the actual PRM input contains the full web interface. Based on this augmented context, the PRM generates a reasoning trace to assess whether the proposed action functionally advances the task, culminating in a binary step-level reward.

3.3 Process Reward Model Grading

A primary limitation of rule-based step rewards is their dependence on golden reference actions. Such rewards assign positive feedback only when the proposed action matches an expert action in action type, target element, and textual input. Consequently, they *can only be applied to states from successful trajectories* where verified reference actions are available, and may penalize valid alternative actions different from the demonstration. This makes rule-based rewards data-inefficient and prevents the agent from learning from failure states.

PRO-CUA replaces these static heuristics with PRM grading. Instead of checking exact agreement with a predefined action, the PRM evaluates whether a proposed action functionally advances the task under the current state. This decouples step-level supervision from expert demonstrations and allows both successful and failed on-policy trajectories to contribute training states.

As illustrated in Figure 2, we formulate the PRM evaluation as a multimodal reasoning task. For any given step, the PRM is provided with a comprehensive context: the task instruction, the action history, and the agent’s proposed current action. Since the candidate action is not executed in the live environment, the PRM does not observe the next state. To ground the evaluation, we annotate the screenshot at the target coordinates of the proposed action, allowing the PRM to identify the intended UI element. The PRM then reasons about whether the proposed action is visually grounded, non-redundant, and useful for task progress, and

outputs a binary step-level reward. The detailed grading prompt is in Appendix F.

Because the PRM is used as a training signal rather than an inference-time controller, PRO-CUA does not require perfectly calibrated rewards. GRPO distinguishes acceptable from unacceptable candidates within a sampled group, and the resulting updates aggregate reward signals across many states, sampled actions, and optimization steps. This makes the training process more tolerant to PRM noise than conventional test-time Best-of-N selection (Chae et al., 2026; Zhang et al., 2026), where a single incorrect PRM preference can directly determine the executed action.

3.4 Policy Optimization

We optimize the policy using the step-level GRPO objective (Equation (1)). Unlike Filtered Behavior Cloning (FBC), which relies on exactly following expert demonstrations, GRPO offers two advantages. First, by relying on the PRM for functional evaluation rather than strict imitation, it preserves reasoning diversity, allowing the agent to freely explore novel thought paths as long as the resulting action is rewarded. Second, the group relative advantage naturally acts as a dynamic curriculum. Because advantages are mean-centered within the sampled group, gradients for trivial states wash out to zero, while difficult states (where most sampled actions fail) generate disproportionately high positive advantages for successful rollouts, focusing optimization where the policy struggles most.

4 Experiments

4.1 Setup

Rollout We use Qwen3-VL-4B-Instruct and Qwen3-VL-8B-Instruct (Bai et al., 2025b) as base models. Following Shen et al. (2025), we use their synthetic task set derived from WebVoyager. At each training iteration, we sample 256 tasks and roll out the current policy to collect on-policy trajectories, running 10 iterations in total. During rollout, we cap the maximum trajectory length at 20 steps, as most tasks are designed to be completed within this horizon, and use a temperature of 1.0 to encourage exploration.

Baselines We compare PRO-CUA with two baseline training algorithms. Both baselines follow the same iterative setting: at each iteration, the current policy interacts with the environment to collect trajectories and obtain task-level outcome rewards.

The collected data is then filtered or converted into training examples as follows: **Filtered Behavior Cloning (FBC)** keeps only successful trajectories and trains the policy on their observed thought-action sequences using SFT. **Step-level RL with rule-based rewards** retains states from successful trajectories and treats the corresponding executed actions as golden references. For each retained state, it samples G candidate thought-action pairs from the current policy, assigns rule-based rewards by comparing each candidate action with the reference action, and optimizes the policy with GRPO. In both baselines, failed trajectories are discarded and do not contribute to training.

Training For FBC experiments, we use LlamaFactory (Zheng et al., 2024) for training with a learning rate of $1e-5$ for 2 epochs. For RL experiments, we use verl (Sheng et al., 2025) with a learning rate of $5e-6$ for 1 epoch. All experiments are conducted on NVIDIA A6000 GPUs with 48G memory. We use a constant learning rate for RL training. Unlike standard offline training, our setting alternates between live environment rollouts and policy optimization, and the number of optimization steps in each iteration depends on the dynamically collected trajectories and their lengths. Therefore, the total number of updates is not known before rollout, making a pre-specified cosine decay schedule less natural. We instead use a constant learning rate for all RL methods to ensure a simple and consistent comparison across iterations.

Evaluation We evaluate PRO-CUA on three online web benchmarks: WebVoyager (He et al., 2024a), Mind2Web-Live (Pan et al., 2024), and Online Mind2Web (Xue et al., 2025). For each task, we allow the agent to take up to 30 steps. Due to website updates, some domains have introduced anti-scraping mechanisms, anti-bot checks, or access restrictions that were not present during benchmark construction. Following common practice for live web evaluation, we exclude domains that cannot be reliably accessed by automated agents, with details provided in Appendix A. Following He et al. (2024a), we provide the full agent trajectory and corresponding screenshots to GPT-5 (Singh et al., 2025), which serves as an automatic evaluator for task success.

4.2 Reliability of the PRM Reward Signal

A key question for PRO-CUA is whether PRM feedback provides a useful training signal compared

Table 1: **Comparison of reward sources for step-level RL.** Under the same successful-trajectory training subset, PRM-based rewards outperform rule-based rewards, suggesting that visually grounded PRM feedback provides useful supervision for policy optimization.

Method	Reward Type	Success Rate
Base model	/	27.5
Step-RL	Rule-based	34.7
Step-RL	Qwen3-VL-4B PRM	36.6
Step-RL	GPT5-mini PRM	36.8

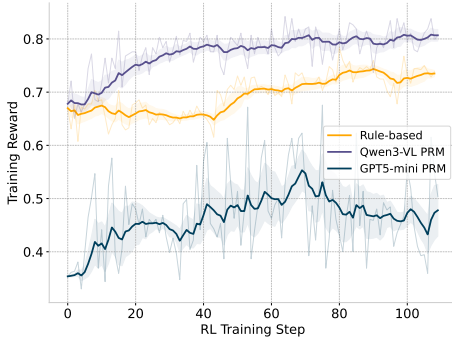


Figure 3: **Step-level rewards assigned during training** with moving average. GPT-5-mini assigns more conservative rewards, while Qwen3-VL-4B is more lenient on average. Despite this calibration gap, both PRMs achieve similar downstream policy performance, suggesting that GRPO is robust to differences in reward strictness through group normalization.

with traditional rule-based rewards. To isolate the effect of the reward source, we conduct a controlled ablation on WebVoyager where all methods train on the same subset of states from successful trajectories. This setting favors rule-based rewards, since golden reference actions are available and failed trajectories are excluded. We compare rule-based rewards with two visually grounded PRMs, Qwen3-VL-4B and GPT-5-mini. As shown in Table 1, both PRM-based variants outperform the rule-based baseline, indicating that PRM feedback can provide effective step-level supervision even under the same data regime as rule-based rewards.

Furthermore, we observe that the lightweight Qwen3-VL-4B PRM achieves similar downstream performance to GPT-5-mini. To better understand this result, we plot the moving average of step-level rewards assigned during optimization. As shown in Figure 3, the two PRMs exhibit substantially different reward calibration: GPT-5-mini is more conservative, with an average reward around 0.5, whereas Qwen3-VL-4B is more lenient, with an average reward above 0.7. One possible explanation is eval-

uator pedantry (Zheng et al., 2023; Li et al., 2024), a phenomenon often observed in strong LLM-as-a-judge systems, where valid exploratory actions may be penalized for deviating from the evaluator’s internal preference or stylistic prior.

Importantly, despite calibration differences, both PRMs lead to comparable final policy performance. This suggests that PRO-CUA does not require perfectly calibrated absolute rewards. Since GRPO computes mean-centered advantages within each sampled group, binary PRM feedback only needs to provide useful local discrimination between acceptable and unacceptable candidate actions. The resulting updates aggregate these signals across many states, sampled candidates, and optimization steps, making the training process robust to differences in reward strictness across PRM backbones.

4.3 Main Results

Benchmark evaluation Table 2 reports the performance on three online web benchmarks. We include both representative open CUA models trained with large-scale expert or closed data, and controlled baselines initialized from the same Qwen3-VL backbones. Compared with prior open models, PRO-CUA is self-evolving: it improves the base policy through on-policy rollouts and PRM-guided step-level RL without relying on external expert demonstrations. Despite this weaker supervision assumption, PRO-CUA achieves competitive performance with expert-trained systems, while substantially improving over its own base models and controlled training baselines.

Within the controlled comparison, PRO-CUA achieves the strongest overall performance across both model sizes, with particularly large gains on WebVoyager and Mind2Web-Live. For the 4B model, PRO-CUA improves over FBC by 12.7% on WebVoyager and 8.3% on Mind2Web-Live in success rate, and improves over rule-based Step-RL by 7.7% and 6.9%, respectively. The gains remain consistent for the 8B model, where PRO-CUA outperforms rule-based Step-RL by 9.4% on WebVoyager and 5.6% on Mind2Web-Live. These results suggest that process-reward-guided step-level RL substantially improves the agent’s ability to interact with dynamic web environments. The improvement is especially pronounced on WebVoyager, likely because its web-domain distribution is closest to our training queries.

Notably, PRO-CUA consistently improves over rule-based Step-RL despite both methods perform-

Table 2: **Task success rates on three online web benchmarks.** We compare PRO-CUA with open CUA models trained on large-scale expert or closed data, and with controlled baselines using Qwen3-VL backbones. PRO-CUA improves the base policy through iterative on-policy self-evolution without relying on expert demonstrations.

Training Paradigm	Method	Ext. Expert Steps	WebVoyager	Mind2Web-Live	OnlineMind2Web
External expert / closed data	UI-TARS-1.5-7B	Closed data	30.3	18.1	14.6
	WebSTAR-7B	100K	47.0	17.0	17.0
	WebSTAR-32B	100K	53.5	20.4	23.8
	GUI-Libra-4B	81K	–	–	20.0
	GUI-Libra-8B	81K	–	–	19.3
Self-evolving 4B	Qwen3-VL-4B-Instruct	0	27.5	18.1	16.7
	FBC	0	29.7	26.4	23.7
	Rule-based Step-RL	0	34.7	27.8	29.9
	PRO-CUA	0	42.4	34.7	28.8
Self-evolving 8B	Qwen3-VL-8B-Instruct	0	25.6	20.8	12.2
	FBC	0	31.8	23.6	26.9
	Rule-based Step-RL	0	33.8	25.0	26.2
	PRO-CUA-8	0	43.2	30.6	28.2

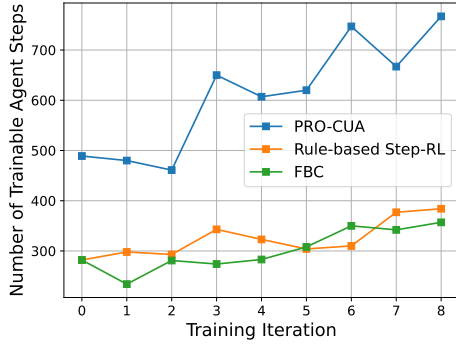


Figure 4: **Data utilization across training iterations.** PRO-CUA consistently yields more usable step-level training data than FBC and rule-based Step-RL because process rewards allow learning from both successful and failed finished trajectories, while the baselines rely on successful rollouts.

ing step-level reinforcement learning with GRPO. This suggests that the key advantage of PRO-CUA comes from the reward source rather than the optimization objective. Rule-based rewards provide reliable supervision only under the existence of golden answers, while our PRM grading is capable of evaluating intermediate steps across both successful and failed trajectories. We analyze this advantage in more detail below.

Data utilization One key advantage of PRO-CUA is its significantly improved data utilization by learning from both successful and failed trajectories, bypassing the strict bottlenecks of only successful trajectories. To illustrate this, we further compare the data utilization of different training paradigms. In PRO-CUA, we apply a lightweight filtering strategy to remove redundant states. In particular, agents may occasionally get stuck on the

same page for many steps, producing highly repetitive states and actions that provide little additional training signal. To reduce such redundancy, we retain only states from *finished* trajectories, where the agent terminates the task and reports an answer within the maximum step budget. Importantly, finished trajectories are not necessarily successful: the final answer may still be incorrect.

Figure 4 shows the number of deployable step-level training examples at each iteration after filtering. PRO-CUA consistently yields substantially more deployable training steps than FBC and rule-based Step-RL. This is because both baselines rely on successful trajectories, while PRO-CUA can learn from both successful and failed trajectories by assigning process rewards to intermediate steps. This enables PRO-CUA to convert a larger fraction of on-policy interactions into trainable supervision, leading to better data utilization throughout iterative training.

5 Related Works

Reinforcement learning for computer use agents Filtered Behavior Cloning (FBC) (He et al., 2026; Xu et al., 2025; Bai et al., 2026) is the prevailing training paradigm for computer use agents. FBC first collects candidate trajectories from human annotators, stronger teacher models, or the agent itself, filters them by final task success, and then trains the policy with SFT on the retained successful trajectories. Recent work has explored RL for GUI and device-control agents to address the limitations of static imitation learning in dynamic environments. DigiRL (Bai et al., 2024) uses offline-to-online RL with advantage-weighted

regression, where the actor is updated by imitating filtered high-advantage actions. Digi-Q (Bai et al., 2025a) learns a Q-function from offline interaction data and extracts a policy by imitating the highest-scored candidate action. These methods go beyond standard behavior cloning, but they still rely on outcome-derived value estimates. For long-horizon GUI tasks, such outcome rewards are sparse and delayed, making it difficult to assign credit to individual intermediate actions.

This motivates recent work on step-level RL for GUI agents. GUI-R1 (Luo et al., 2025) and UI-R1 (Lu et al., 2026) apply R1-style rule-based RL for GUI action prediction, while GUI-Libra (Yang et al., 2026) improves training under partially verifiable GUI rewards. However, the reward design of those methods still depends on verifiable rules or reference actions. Moreover, they typically construct training states from teacher-generated trajectories, which are often mismatched with the states encountered by the target policy. In contrast, PRO-CUA collects states on-policy from the current agent and uses visually grounded PRM feedback to score candidate thought-action pairs at those states. This preserves the credit-assignment benefit of step-level RL while reducing off-policy distribution shift and avoiding the rigidity of rule-based rewards, enabling learning from both successful and failed trajectories.

Process reward models PRMs have achieved widespread success in reasoning-intensive domains such as mathematics (Lightman et al., 2024; Wang et al., 2024; Zhang et al., 2025b) and code generation (Dai et al., 2024). They are commonly used in two ways: *test-time scaling*, where a PRM ranks sampled solutions or actions through Best-of- N selection or reward-guided search (Wang et al., 2024, 2025b; Xiong et al., 2025), and *training-time supervision*, where PRM scores serve as intermediate rewards for reinforcement learning (Setlur et al., 2025; Zhang et al., 2025a). Computer use tasks are structurally suited for step-level supervision because GUI navigation offers natively atomic steps (discrete clicks/typing), and a single erroneous action can derail the entire trajectory (e.g., closed tabs or failed verifications). However, test-time scaling is particularly costly in this setting: each candidate action may require environment interaction, screenshot processing, and PRM evaluation, making search-style methods substantially more expensive than single-policy execution.

Recent work has developed PRMs for web and computer-use agents. WebShepherd (Chae et al., 2026) trains a checklist-based PRM for web navigation, while WebArbiter (Zhang et al., 2026) improves it with principle-guided reasoning. However, these works primarily use PRMs as inference-time evaluators for action selection. This does not internalize the reward signal into the policy and is sensitive to PRM noise, since the agent directly executes the action preferred by the reward model. As CUARewardBench (Lin et al., 2025) suggests, even proprietary models remain primitive in step-level judging for CUA tasks, and most open-sourced PRMs are distilled from proprietary teachers rather than serving as reliable ground-truth evaluators. In contrast, PRO-CUA uses PRM as a source of training rewards in iterative on-policy RL. By aggregating noisy step-level signals across many states, sampled actions, and optimization updates, PRO-CUA is more robust than committing to a single PRM-selected action at inference time, as supported by our validation that Qwen3-VL-4B performs comparably to GPT-5-mini. Compared with offline filtering methods such as WebSTAR (He et al., 2026), PRO-CUA directly optimizes the policy with PRM scores, enabling both successful and failed on-policy trajectories to provide supervision.

6 Conclusion

We introduced PRO-CUA, an iterative step-level reinforcement learning framework for training computer use agents with process rewards. PRO-CUA decouples slow live environment interaction from policy optimization by first collecting on-policy states from the current agent and then optimizing the policy over PRM-graded candidate thought-action pairs. This design addresses two central bottlenecks in existing CUA training: the distribution shift caused by relying on teacher-generated states, and the limited data utilization caused by rule-based rewards that require successful trajectories or golden actions. By using flexible PRM feedback as step-level rewards, PRO-CUA can learn from both successful and failed finished trajectories and convert a larger fraction of on-policy interactions into trainable supervision. Experiments on three online web benchmarks show that PRO-CUA consistently improves over filtered behavior cloning and rule-based Step-RL, demonstrating the promise of process-reward-guided on-policy training for building more capable computer use agents.

Limitations

While PRO-CUA provides an effective framework for process-reward-guided training of computer use agents, several directions remain beyond the scope of this work.

First, we follow the standard CUA setup where the policy observes the task instruction, action history, and the current screenshot. We do not incorporate more advanced harnessing strategies such as long-term memory, retrieval, or explicit context engineering. These components are complementary to our training framework and could further improve performance on tasks that require long-range information tracking.

Second, our experiments only focus on web-based computer-use tasks. This setting provides a realistic and widely used testbed for CUA research because web environments are dynamic, visually diverse, and require long-horizon interaction. However, computer use agents are broader than web navigation alone, and many practical applications involve desktop software and mobile apps. These environments may introduce different interaction patterns, interface conventions, safety constraints, and state representations. While PRO-CUA is designed as a general training framework and does not rely on web-specific assumptions beyond the environment interface, validating its effectiveness beyond web benchmarks remains an important direction for future work.

References

- Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. 2025. Agent s: An open agentic framework that uses computers like a human. In *The Thirteenth International Conference on Learning Representations*.
- Anthropic. 2024. Developing a computer use model. <https://www.anthropic.com/news/developing-computer-use>.
- Hao Bai, Alexey Taymanov, Tong Zhang, Aviral Kumar, and Spencer Whitehead. 2026. Webgym: Scaling training environments for visual web agents with realistic tasks. *arXiv preprint arXiv:2601.02439*.
- Hao Bai, Yifei Zhou, Li Erran Li, Sergey Levine, and Aviral Kumar. 2025a. Digi-q: Learning VLM q-value functions for training device-control agents. In *The Thirteenth International Conference on Learning Representations*.
- Hao Bai, Yifei Zhou, Jiayi Pan, Mert Cemri, Alane Suhr, Sergey Levine, and Aviral Kumar. 2024. Digi-girl: Training in-the-wild device-control agents with

autonomous reinforcement learning. *Advances in Neural Information Processing Systems*, 37:12461–12495.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, and 1 others. 2025b. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*.

Rogério Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, and 1 others. 2025. Windows agent arena: Evaluating multi-modal os agents at scale. In *Forty-second International Conference on Machine Learning*.

Hyungjoo Chae, Sunghwan Kim, Junhee Cho, Seungone Kim, Seungjun Moon, Gyeom Hwangbo, Dongha Lim, Minjin Kim, Yeonjun Hwang, Minju Gwak, Dongwook Choi, Minseok Kang, Gwanhoon Im, ByeongUng Cho, Hyojun Kim, Jun Hee Han, Taeyoon Kwon, Minju Kim, Beong woo Kwak, and 2 others. 2026. Web-shepherd: Advancing PRMs for reinforcing web agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Ning Dai, Zheng Wu, Renjie Zheng, Ziyun Wei, Wenlei Shi, Xing Jin, Guanlin Liu, Chen Dun, Liang Huang, and Lin Yan. 2024. Process supervision-guided policy optimization for code generation. *arXiv preprint arXiv:2410.17621*.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024a. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Hongming Zhang, Tianqing Fang, Zhenzhong Lan, and Dong Yu. 2024b. Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization. *arXiv preprint arXiv:2410.19609*.

Yifei He, Pranit Chawla, Yaser Souri, Subhojit Som, and Xia Song. 2026. Webstar: Scalable data synthesis for computer use agents with step-level filtering. *Preprint*, arXiv:2512.10962.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.

747	Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llm-as-judges: a comprehensive survey on llm-based evaluation methods. <i>arXiv preprint arXiv:2412.05579</i> .	802
748		803
749		804
750		805
751		
752	Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s verify step by step. In <i>International Conference on Learning Representations</i> , volume 2024, pages 39578–39601.	806
753		807
754		808
755		809
756		810
757		811
758	Haojia Lin, Xiaoyu Tan, Yulei Qin, Zihan Xu, Yuchen Shi, Zongyi Li, Gang Li, Shaofei Cai, Siqi Cai, Chaoyou Fu, and 1 others. 2025. Cuareward-bench: A benchmark for evaluating reward models on computer-using agent. <i>arXiv preprint arXiv:2510.18596</i> .	812
759		813
760		814
761		815
762		816
763		
764	Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, and 1 others. 2026. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In <i>Proceedings of the AAAI Conference on Artificial Intelligence</i> , volume 40, pages 17608–17616.	817
765		818
766		819
767		820
768		821
769		822
770		
771	Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. 2025. Gui-r1: A generalist r1-style vision-language action model for gui agents. <i>arXiv preprint arXiv:2504.10458</i> .	823
772		824
773		825
774		826
775		827
776		828
777		829
778		
779	OpenAI. 2025. Introducing operator. https://openai.com/index/introducing-operator/ .	830
780		831
781		832
782	Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and 1 others. 2024. Web-canvas: Benchmarking web agents in online environments. <i>arXiv preprint arXiv:2406.12373</i> .	833
783		834
784		
785	Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, and 1 others. 2025. Ui-tars: Pioneering automated gui interaction with native agents. <i>arXiv preprint arXiv:2501.12326</i> .	835
786		836
787		837
788		838
789		839
790		
791	Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. 2011. A reduction of imitation learning and structured prediction to no-regret online learning. In <i>Proceedings of the fourteenth international conference on artificial intelligence and statistics</i> , pages 627–635. JMLR Workshop and Conference Proceedings.	840
792		841
793		842
794		843
795		844
796	Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. 2025. Rewarding progress: Scaling automated process verifiers for llm reasoning. In <i>International Conference on Learning Representations</i> , volume 2025, pages 60808–60838.	845
797		846
798		847
799		848
800		849
801		850
		851
	Jiang, Tong Zhang, Ameet Talwalkar, and 1 others. 2025. Thinking vs. doing: Agents that reason by scaling test-time interaction. <i>arXiv preprint arXiv:2506.07976</i> .	852
		853
		854
		855
	Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In <i>Proceedings of the Twentieth European Conference on Computer Systems</i> , pages 1279–1297.	856
		857
	Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, and 1 others. 2025. Openai gpt-5 system card. <i>arXiv preprint arXiv:2601.03267</i> .	
	Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, and 1 others. 2025a. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. <i>arXiv preprint arXiv:2509.02544</i> .	
	Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In <i>Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 9426–9439.	
	Weiyun Wang, Zhangwei Gao, Lianjie Chen, Zhe Chen, Jinguo Zhu, Xiangyu Zhao, Yangzhou Liu, Yue Cao, Shenglong Ye, Xizhou Zhu, and 1 others. 2025b. Visualprm: An effective process reward model for multimodal reasoning. <i>arXiv preprint arXiv:2503.10291</i> .	
	Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, and 1 others. 2025c. Opencua: Open foundations for computer-use agents. <i>arXiv preprint arXiv:2508.09123</i> .	
	Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, and 1 others. 2025. Gui-actor: Coordinate-free visual grounding for gui agents. <i>arXiv preprint arXiv:2506.03143</i> .	
	Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. <i>Advances in Neural Information Processing Systems</i> , 37:52040–52094.	
	Tao Xiong, Xavier Hu, Yurun Chen, Yuhang Liu, Changqiao Wu, Pengzhi Gao, Wei Liu, Jian Luan, and Shengyu Zhang. 2025. Gui-pra: Process reward agent for gui tasks. <i>arXiv preprint arXiv:2509.23263</i> .	
	Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and	

Caiming Xiong. 2025. Aguviz: Unified pure vision agents for autonomous gui interaction. In *Forty-second International Conference on Machine Learning*.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382*.

Rui Yang, Qianhui Wu, Zhaoyang Wang, Hanyang Chen, Ke Yang, Hao Cheng, Huaxiu Yao, Baoling Peng, Huan Zhang, Jianfeng Gao, and 1 others. 2026. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Yao Zhang, Shijie Tang, Zeyu Li, Zhen Han, and Volker Tresp. 2026. *Webbarbiter: A generative reasoning process reward model for web agents*. In *The Fourteenth International Conference on Learning Representations*.

Yuxin Zhang, Meihao Fan, Ju Fan, Mingyang Yi, Yuyu Luo, Jian Tan, and Guoliang Li. 2025a. Reward-sql: Boosting text-to-sql via stepwise reasoning and process-supervised rewards. *arXiv preprint arXiv:2505.04671*.

Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingen Zhou, and Junyang Lin. 2025b. The lessons of developing process reward models in mathematical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10495–10516.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.

Yaowei Zheng, Richong Zhang, Junhao Zhang, YeYanhan YeYanhan, and Zheyang Luo. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 400–410.

A Evaluation Details

In Table 3, we detail specific websites that currently employ strict anti-bot mechanisms, preventing automated access. Consequently, we exclude all tasks requiring interaction with these domains from both our training and evaluation sets. It is important to note that these accessibility issues reflect the state

of these platforms at the time of our experiments (May 2026); future updates may resolve these barriers and render the tasks viable again. Additionally, we omit tasks associated with Google, GitHub, and Allrecipes from the WebVoyager evaluation, as these platforms mandate rigorous human verification that falls outside the scope of our automated agent.

B Experimental Details

For the WebVoyager results in Table 2, we directly from the original WebSTAR paper (He et al., 2026). Since Allrecipes and GitHub are deprecated as mentioned above, we compute the average performance for UI-TARS-1.5-7B, WebSTAR-7B and WebSTAR-32B without considering those domains to ensure a fair comparison.

Following the definition in Qwen3-VL (Bai et al., 2025b), we present the action space of the computer use task in Table 4.

C Implementation Details of Rule-Based Rewards

For the rule-based Step-RL baseline, we implement an automated verifier following GUI-Libra (Yang et al., 2026). Each sampled rollout produces a structured prediction string y following the required output format:

$$y = \langle \text{think} \rangle \dots \langle / \text{think} \rangle \langle \text{answer} \rangle a \langle / \text{answer} \rangle,$$

where the $\langle \text{answer} \rangle$ block contains a structured action a that can be parsed into a JSON object: $a = \{\text{action_type}, \text{description}, \text{value}, \text{point_2d}\}$. Here, action_type specifies the primitive GUI operation, value denotes the text input when applicable, and $\text{point_2d} \in \mathbb{R}^2$ denotes the target screen coordinate, or none when the action does not require grounding.

We define the rule-based reward as a weighted combination of a format reward and an action-correctness reward:

$$\tilde{r}(s, a) = w_{\text{fmt}} r_{\text{fmt}} + (1 - w_{\text{fmt}}) r_{\text{acc}},$$

where $w_{\text{fmt}} \in [0, 1]$. In our experiments, we set $w_{\text{fmt}} = 0.1$, so the reward primarily reflects action correctness while still encouraging valid structured outputs.

Format reward The format reward r_{fmt} checks whether the model output can be parsed correctly. We set $r_{\text{fmt}} = 1$ if the output contains valid

Table 3: Websites with access problems in Mind2Web queries.

Error	Websites
Human verification	bbb.org, redfin.com, extraspace.com, seatgeek.com, google.com/shopping, sec.gov, expedia.com, discogs.com, allrecipes.com
Access denied	americashealthrankings.org, adoptapet.com, reddit.com, qatarairways.com, fedex.com, macys.com, gamestop.com, disney.com, kbb.com, accuweather.com
Website issue	nba.com, target.com, phys.org, ups.com, kfc.com, cars.com, bestbuy.com, apartments.com, kohls.com

Table 4: Action space for computer use agents.

Action	Definition
left_click(x,y)	Clicks at coordinates (x, y)
double_click(x,y)	Double-clicks at (x, y)
right_click(x,y)	Right-clicks at (x, y)
mouse_move(x,y)	Move the cursor to (x, y)
left_click_drag(x1,y1,x2,y2)	Drags from (x1, y1) to (x2, y2)
scroll(x,y,dir)	Scrolls at (x, y) in given direction
type(content)	Types text
hotkey(keys)	Presses hotkey
wait()	Pauses 5s
goback()	Go back to the previous page
finished(content)	Complete task with final answer

<think> and <answer> tags and the <answer> block can be parsed into the required JSON schema. Otherwise, $r_{\text{fmt}} = 0$.

Accuracy reward The accuracy reward r_{acc} evaluates whether the predicted action matches the reference action from a successful trajectory. We decompose it into three components:

$$r_{\text{acc}} = r_{\text{type}} \cdot r_{\text{value}} \cdot r_{\text{ground}},$$

where r_{type} evaluates the action type, r_{value} evaluates the textual input, and r_{ground} evaluates coordinate grounding.

First, the action-type reward checks whether the predicted action type matches the reference action type:

$$r_{\text{type}} = \mathbb{1} [\text{action_type}(a) = \text{action_type}(a^*)],$$

where a^* denotes the reference action.

Second, for actions involving text input, the value reward compares the predicted value v with the reference value v^* using word-level F1:

$$r_{\text{value}} = \mathbb{1} [\text{F1}(v, v^*) > 0.5].$$

For actions that do not require a textual value, we set $r_{\text{value}} = 1$.

Third, the grounding reward evaluates whether the predicted coordinate $\mathbf{u}$ falls inside the reference bounding box b^* :

$$r_{\text{ground}} = \mathbb{1} [\mathbf{u} \in b^*].$$

For actions that do not require coordinate grounding, we set $r_{\text{ground}} = 1$.

This rule-based reward provides reliable supervision when a reference action is available. However, it is inherently tied to successful trajectories with golden actions. It may assign low rewards to valid alternative actions that deviate from the reference, and cannot be applied to states from failed trajectories where no verified reference action is available. This limitation motivates our use of PRM-based rewards in PRO-CUA.

D Potential Risks

This paper presents work whose goal is to advance the field of NLP. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

E Licenses

Both WebVoyager and OpenWebVoyager are under Apache-2.0 license. Both Online-Mind2Web Mind2Web-Live are under cc-by-4.0 license.

F Prompt for PRM Grading

PRM grading prompt

You are an expert evaluator grading a Computer-Use Agent. Your role is to evaluate whether the agent's proposed next action is the strictly correct and necessary step to advance the given task. You are provided with:

1. The overarching task instruction.
2. The history of actions taken so far.
3. The CURRENT screenshot (the state immediately BEFORE the proposed action), annotated to show the proposed target of the action.
4. The proposed Thought and Action Code.

The screenshot is an annotated visualization of the proposed action, not a raw screenshot:

- Red marks, arrows, or points indicate where the proposed action is targeting.
- Small index labels and overlay text are part of the annotation.
- Use these annotations to judge whether the proposed action is correctly grounded on the UI.
- Do not confuse the annotation itself with a native page element.

`<task_instruction>` instruction `</task_instruction>`

`<history_actions>` history_actions `</history_actions>`

`<proposed_action>` Step step_index: action_code `</proposed_action>`

Evaluation Criteria You must evaluate the proposed action and output a binary decision: is the action CORRECT or INCORRECT?

An action is INCORRECT if it exhibits ANY of the following flaws:

- Grounding Failure: The code targets the wrong coordinates, a non-existent element, or the wrong input field based on the provided screenshot.
- Hallucination: The agent assumes a state that is not visually present.
- Inefficiency/Redundancy: The action needlessly repeats a past step from the history, performs useless scrolling, or wastes a step without advancing the task.
- Logical Progression Failure: The action executes successfully but does not move the agent closer to the final goal.

An action is CORRECT ONLY if it is visually grounded, mathematically accurate, and actively advances the task toward completion.

Output Format Provide a rigorous step-by-step reflection. You must perform a "mental rollout" to predict the consequences of the action before determining if it facilitates task completion. Think about other alternatives that might result in better outcome than the proposed action, and if there exists such alternative with strictly better outcome, make the action as incorrect. Then, output a strictly valid JSON block.

`<analysis_process>`

1. [Current State Assessment]: What is currently visible on the screen? What is the immediate blocker to completing the task?
2. [Target Verification]: Does the proposed code correctly and accurately target the intended UI element in the screenshot?
3. [Mental Rollout]: If this exact code is executed, what will happen? (e.g., "A dropdown menu will appear," "The page will scroll down," "The text 'shoes' will be typed").
4. [Task Alignment]: Does this predicted outcome meaningfully and efficiently advance the task? Or is it a redundant/wasteful action given the history?
5. [Final Verdict]: Conclude whether the step is Correct or Incorrect.

`</analysis_process>`

`““json`

`"is_correct": boolean,`

`"reflection": "A 1-2 sentence summary of why the action was marked correct or incorrect."`

`”””`